

Sammansatta datatyper

De datatyper som hittills behandlats har varit *primitiva* datatyper, d.v.s. bestått av endast en enda datatyp. Vektorer är en aning mera komplicerade, men de innehåller alltid en mängd av samma datatyp. I C++ kan man skapa *sammansatta* datatyper som är en sammansättning av flera olika datatyper. Varför vill man göra något sådant? Om vi t.ex. vill skapa ett litet enkelt adressregister som bokför personnamn, telefonnummer och kanske en adress, måste vi med våra kunskaper hittills skapa skilda vektorer med strängar för namn, adresser o.s.v. Det vore mera praktiskt att kunna gruppera all information om en person i en enda variabel, och sedan gruppera dessa person-variabler i t.ex. en vektor. Här kommer sammansatta datatyper in. De tillåter programmeraren att skapa datatyper som är sammansättningar av de primitiva (och tidigare definierade sammansatta datatyperna), och på så vis bygga upp just den datatyp som behövs. Denna funktionalitet erbjuds av `struct` i C++. Allmänt ser en `struct` ut på följande sätt:

```
struct strukturnamn {  
    datatyp1 variabelnamn1;  
    datatyp2 variabelnamn2;  
    ...  
    datatypN variabelnamnN;  
};
```

Definitionen börjar alltså med nyckelordet `struct`, följt av det namn som önskas på den nya sammansatta datatypen. Innanför `{` och `};` definieras sedan de variabler som datatypen skall innehålla. I vårt fall kunde vi definiera en datatyp `Person` på följande sätt:

```
struct Person {  
    string Namn;  
    string Adress;  
    int    Telefonnummer;  
};
```

Vår datatyp `Person` kan nu användas som vilken datatyp som helst överallt i vårt program där definitionen är känd. Notera att denna definition inte ännu definierat variabler av vår nya datatyp, detta måste göras separat:

```
Person Ingvar;  
Person Elsa;
```

Variablerna `Ingvar` och `Elsa` kan användas som helt vanliga variabler, skickas som parametrar o.s.v.

Not: I C++ behövs inget nyckelord `struct` då man deklarerar variabler av en sammansatt datatyp, vilket var obligatoriskt i C, där man måste definiera variabeln `Ingvar` som `struct Person Ingvar;`. Detta har ändrats i C++, kanske närmast för att göra `struct` mera lik en klass (se [Kapitel 14](#)).

Scoping

Definitionen på en `struct` har även ett scope, d.v.s. det område i programmet där den är definierad.

Man kan definiera en `struct` nästan var som helst i ett C++-program, och det ger vissa scoping-regler. En `struct` som är definierad inom en viss funktion kan endast användas inom den funktionen. Vill man att datatypen skall kunna användas av alla delar av ett program bör den placeras *externt*, d.v.s. utanför alla funktioner i början av programmet.

```
struct Kund {
    string Namn;
    string Adress;
};

void foobar () {
    // lokal definition, kan endast användas i denna funktion
    struct Rakning {
        string Vara;
        int Pris;
    }

    // definiera variabler av datatyper
    Kund Kl;
    Rakning Spikar, Hammare;
    ...
}

int main () {
    // definiera variabler av datatyper
    Kund Foo, Bar;
    ...
}
```

I exemplet ovan innehåller funktionen `foobar` en egen lokal definition på en `struct`. Denna kan endast användas inom `foobar()`, eftersom dess scope inte sträcker sig till `main()`. Däremot kan `Kund` användas överallt i programmet, eftersom den är definierad utanför (och före) alla funktioner.

Accessera medlemmar

Vi har ingen glädje av `struct`:s om vi inte kan accessera datamedlemmar i en `struct`. Det är dock mycket enkelt att accessera och ändra data. Det görs genom att man avrefererar datamedlemmen med en punkt (`.`). För att t.ex. ge en ny telefonnummer åt medlemmen `Telefonnummer` i variabeln `Ingvar` gör man så här:

```
Person Ingvar;
Ingvar.Telefonnummer = 5556789;
```

Alla datamedlemmar kan accesseras med en punkt. Efter denna avreferering kan de användas precis på samma sätt som datatypen normalt kan användas. I exemplet ovan kan `Ingvar.Telefonnummer` användas precis som en normal `int`.

Då man skapar nya variabler av en `struct` kanske man vill direkt ge ett värde åt alla medlemmar. Då kan man göra enligt följande:

```
Person Ingvar = { "Ingvar Infåit", "DataCity", 5551234 };
```

Värdena sätts innanför `{` och `}` i exakt den ordning som de givits då datatypen `Person` definierats. Första värdet ges åt första medlemmen o.s.v. Vill man ge ett värde direkt är detta den enda möjligheten. Följande är *inte* tillåtet, och kommer att resultera i ett kompilersfel:

```
Person Ingvar;  
Ingvar = { "Ingvar Infåit", "DataCity", 5551234 };
```

Jämföra och tilldela

Att jämföra och tilldela `struct`:s kan vara lite knepigt. Tilldelning enligt följande är inte alltid möjlig:

```
Person Ingvar = { "Ingvar Infåit", "DataCity", 5551234 };  
Person Elsa = Ingvar;
```

Det hela beror på innehållet i `struct`:en. Det som sker i en tilldelning som denna är att kompilatorn kommer att se till att `Elsa` tilldelas alla medlemmar från `Ingvar`. Ovanstående i princip ekvivalent med följande:

```
Person Ingvar = { "Ingvar Infåit", "DataCity", 5551234 };  
Person Elsa;  
Elsa.Namn = Ingvar.Namn;  
Elsa.Adress = Ingvar.Adress;  
Elsa.Telefonnummer = Ingvar.Telefonnummer;
```

För vårt exempel fungerar tilldelningen fint, eftersom tilldelningsoperatoren `=` är definierad för alla datatyper som används i `Person`. I alla sammanhang är det inte så, speciellt om olika främmande `struct`:s används. Om `=` inte är definierad får man i vissa fall resultat som inte är önskvärda. Man kan då explicit definiera en `=`-operator för `struct`:en, men i sådana fall är det i alla fall mera praktiskt att använda en klass (se [Kapitel 14](#)). Så länge som man endast använder primitiva datatyper i en `struct` kan man använda normal tilldelning.

Jämförelse av sammansatta datatyper är ännu svårare. Där kan man i normala fall aldrig direkt använda sig av operatoren `==` för att åstadkomma en jämförelse, utan man måste manuellt jämföra varje medlem. Detta kan vara ganska arbetsdrygt och fult om det är frågan om en stor datatyp. I kapitlet [Kapitel 21](#) redogörs för hur man kan definiera operatoren `==` för klasser, men det samma kan även appliceras på `struct`:ar.

Sammansatta datatyper som parametrar

Man kan givetvis skicka sammansatta datatyper som parametrar till funktioner. De fungerar precis som vilka datatyper som helst. Vill man kunna ändra på en `struct` som skickats som parameter bör man skicka en pekare eller använda referensparametrar (`&`). Exemplet nedan läser in en `Produkt` och skriver ut denna:

```
#include <iostream>  
#include <string>  
  
struct Produkt {  
    string Namn;  
    float Pris;  
    int Antal;  
};  
  
void nyProdukt (Produkt & P) {  
    // läs in data i produktens medlemmar  
    cout << "Namn : ";  
    cin >> P.Namn;  
    cout << "Pris : ";
```

```

    cin >> P.Pris;
    cout << "Antal: ";
    cin >> P.Antal;
}

void skrivUt (Produkt * P) {
    // skriv ut info om produkten
    cout << endl;
    cout << "Information om produkt: " << P->Namn << endl;
    cout << "    pris: " << (*P).Pris << endl;
    cout << "    antal: " << P->Antal << endl;
}

int main () {

    Produkt Ny;

    // läs in en produkt
    nyProdukt ( Ny );

    // skriv ut på skärmen
    skrivUt ( &Ny );
}

```

Körning av programmet kan t.ex. se ut på följande sätt:

```

% Struct4
Namn : C++-kompilator
Pris  : 19.90
Antal: 3

Information om produkt:
    namn: C++-kompilator
    pris: 19.9
    antal: 3
%

```

Här har vi nu använt oss av två sätt att skicka en `struct` som parameter. Det tredje sättet är det normala, d.v.s. helt som en normal variabel utan `&` eller pekare. Vi kunde använt det sättet för funktionen `skrivUt()`, eftersom den inte behöver ändra värden i `P`, men för `nyProdukt()` måste vi använda någondera sättet som tillåter oss att ändra värden i `P`. Notera att om man använder `&` kan man direkt referera till medlemmar med en `.`, men använder man pekare är det lite mera knepigt. Det lättare sättet då man använder pekare är att använda sig av *pilnotationen*. En `->` används för att avreferera en pekare som pekar på en `struct` (eller en klass) och accessera en medlem. Om man inte använder `->` bör man först avreferera pekaren med `*`, och därefter avreferera medlemmen med en punkt (`.`). Använder man `*`-metoden bör man notera att `.` har högre precedens än `*`, och evalueras således först, vilket ju inte är vad vi vill. Vi vill först evaluera `*` och därefter `..` Vi måste således använda en parentes för att tvinga fram en viss precedensordning. Med tanke på detta är `->` att föredra. Notera att `->` endast kan användas tillsammans med pekare.

Effektivitet vid parameteröverföring

Hittills i våra program och exempel har vi inte behandlat effektivitet speciellt mycket. Då man hanterar sammansatta datatyper finns det en sak man bör notera, speciellt om man skickar dylika som parametrar till olika funktioner. Om man skickar en `struct` som en vanlig parameter (ej pekare och utan `&`) så kommer kompilatorn att kopiera hela innehållet i `struct`:en till mottagarfunktionen. I våra små exempel är det inte mycket som behöver kopieras, men i ett verkligt system kan man ha tiotals medlemmar i dylika datatyper. Om varje funktionsanrop innebär att stora mängder data

måste kopieras kan ett program enkelt bli långsamt. Lösningen till detta problem är att undvika att kopiera så mycket data! Man kan åstadkomma detta genom att t.ex. använda pekare eller referensparametrar. I båda fallen skickas endast adressen på variablen i fråga, och stora inbesparningar kan uppnås. Om & används kan mottagaren använda parametern precis på samma sätt som om den vore skickad på normalt sätt. Således rekommenderas att & används för överföring av sammansatta datatyper.

Gäller samma resonemang t.ex. vektorer, som ju kan vara väldigt stora? Nej, det gör det inte, eftersom kompilatorn alltid i kompileringsskedet ersätter vektor-parametrar med pekare. Därmed skickas alltid endast adressen på första elementet till den mottagande funktionen.

[Förutgående](#)

Flerdimensionella vektorer

[Hem](#)[Upp](#)[Nästa](#)

Enumereringar